COEP TECHNOLOGICAL UNIVERSITY

Department of Computer Science and Engineering

Mini Projects — Guidelines to Students

Mini Project 9 · Graph Algorithms Explorer

FOUNDATION-FOCUSED

An interactive toolkit that builds graphs and computes their structural properties and classic algorithms.

A Foundations-First Framework · August 2026

1. What a Mini Project Is — and What We Expect

A mini project is a single working system that three groups of 4 students each, develop across the semester, where several of your foundational subjects meet inside one artefact. The idea is simple: you build the core concepts yourself instead of importing a library that hides them. This is what we mean by foundations-first — the project is a reason to understand Python and OOP, data structures and algorithms, discrete structures, digital logic, and microprocessors from the inside.

Two habits matter more than anything else. First, **go deep, not wide**: your project is split into three worklets of five tasks each, so you finish one solid layer before the next. Second, **be ready to explain why**, not just show that it runs — every student answers a viva individually, so understanding is assessed per person, not per group. This document is your brief for Mini Project 9, the Graph Algorithms Explorer.

2. Mini Project 9 — Overview

An interactive toolkit that builds graphs and runs the classic graph algorithms on them. It is a foundation-focused project that turns graph theory into working code: traversal, shortest paths, spanning trees and connectivity, each backed by the right representation and data structure and verified against first principles.

What you will build

- Create and edit graphs (directed/undirected, weighted).
- Traversals, shortest paths, spanning trees, connectivity, and cycles.
- Structural metrics: degree distribution, centrality, components, colouring.
- Visualise results on sample networks and mazes.

Foundational courses it covers

- Discrete Structures & Computational Thinking (graph theory).
- Data Structures & Algorithms (graph representations and algorithms).
- Python Programming & OOP (graph model).

Technologies

Python (with a plotting/graph library), or C++.

3. Architecture & Design — How to Think About the Build

Before writing code, decide on the objects, the data structures, and the layers. Good design here is mostly classic computer science, applied deliberately. The exclusions are the point: you choose your language, but the concepts below are built by you, not pulled from a package.

Design area	For Mini Project 9
Algorithms	BFS, DFS, and connected components; Dijkstra and Bellman–Ford shortest paths; Kruskal / Prim minimum spanning tree; topological sort and cycle detection; graph colouring and bipartite test.
Data structures	Adjacency list and adjacency matrix; a priority queue / min-heap (Dijkstra); union-find (disjoint sets); queues (BFS) and stacks (DFS); hash maps for visited / distances.
Design concepts	A layered design (graph model, algorithms, visualization); a strategy pattern for interchangeable algorithms; representation separated from traversal; pluggable graph representations.
Exclusions (build it yourself)	No graph libraries (NetworkX, igraph), no graph databases, no web frameworks or cloud services, and no machine-learning or AI libraries. Python or C++ is your language.

4. The Three Worklets

The project is delivered through three worklets, wherein the worklets run parallely. One group of 4 students works on a worklet. The worklets are cumulative: each makes use of the artefacts produced by the other, taking the student from core representation to a complete, demonstrable system. *Interface definitions for each worklet are therefore crucial, and must be defined as part of the initial design.* The three worklets finally integrate to create a working demonstration.

Worklet 1 — Graph model, traversal & connectivity (DSA + Discrete)

Build the graph model in two representations, then implement traversal and connectivity so the structure of a graph is visible.

- Implement adjacency-list and adjacency-matrix representations (directed/undirected/weighted).
- Build editing operations and load graphs from an edge-list file.
- Implement BFS and DFS with visit ordering.
- Find connected (and strongly connected) components; detect cycles.
- Compute a topological order for a DAG; test bipartiteness by 2-colouring.

Worklet 2 — Weighted-graph algorithms

Add the weighted-graph algorithms — shortest paths and minimum spanning trees — and verify them against brute force.

- Implement Dijkstra's shortest path with a priority queue.
- Implement Bellman–Ford and detect negative cycles.
- Implement a minimum spanning tree (Kruskal or Prim) with union-find.
- Verify results against brute force on small graphs.
- Compare the space/time trade-offs of the two representations.

Worklet 3 — Structural analysis, visualisation & evaluation

Analyse and visualise: structural metrics, colouring, mazes, animated traversals, and benchmarks as the graph grows.

- Compute degree distributions and a centrality measure.
- Greedily colour the graph; generate and solve mazes as graphs.
- Render graphs and highlight algorithm outputs (paths, trees).
- Animate a traversal step by step.
- Run on a real-ish dataset and benchmark as size grows.

How your understanding is checked

At each review you will be asked viva-style questions like these — they check that you understand the design, not just that the code runs:

- When is an adjacency matrix better than a list, and why?
- Why does Dijkstra fail with negative edges, and what does Bellman–Ford do instead?

- Prove a graph is bipartite iff it has no odd cycle, and show your test detects it.
- What does union-find give Kruskal, and what is its near-constant cost?
- Trace BFS from a node and explain why it yields shortest paths in an unweighted graph.

5. How You Will Work — Project Execution

First, all groups within the mini project work together to agree the requirements, high-level design, and interface definitions across the worklets, within 2-3 weeks. Then teams take individual worklets and build them independently, meeting a reviewer every two weeks. Finally, the groups come back together to consolidate and integrate the worklets into one system, ending with a demonstration and an individual viva.

- **Weekly effort —** 6 contact hours per week — 2 hours in the Software Engineering lab plus 4 hours on a working Saturday.
- **Reviews —** a design review in week 3, then a worklet review roughly every two weeks.
- **Individual accountability —** the final viva is per student; know your own contribution and the whole system.
- **Working in your group —** within each worklet, rotate who presents; agree the shared interfaces early (the graph representation (list vs matrix), the edge-list file format, and the algorithm / result interface); keep one shared repository with small, frequent commits; and review each other's worklets, because if you can explain a teammate's design you are ready for the viva.

Getting started — your first two weeks

Use the opening fortnight to agree the shared design before anyone writes the hard code. A little planning here prevents painful re-integration later.

- Agree the domain model together: the core objects, their fields, and the operations each module must expose.
- Fix the interfaces between worklets early — the graph representation (list vs matrix), the edge-list file format, and the algorithm / result interface.
- Write down the design decisions and their trade-offs before committing to them.
- Set up one shared repository, a build that runs for all four members, and a tiny end-to-end test.
- Assign a clear owner to each worklet and agree who presents at the week-3 design review.

6. Timeline — 14 Weeks

The project runs on a common 14-week schedule. You put in 6 contact hours a week — 2 hours in the SE lab plus 4 hours of mini-project work. The letter (E) marks a scheduled evaluation point: about 1.5 hours per project (30 minutes per worklet), presented by one student in rotation, and carrying the weight shown.

When	Phase	Milestone / evaluation
W1	Orientation, group & project allocation, setup	Signed project plan.
W1–2	Requirements & high-level design (all groups)	Requirements + high-level design.
W3	Requirements & design evaluation	Requirements & design review — (E) 29 Aug · 20%.
W4–10	Worklet 1, 2, 3 development (independent teams)	Working reviews + builds + demos for each worklet independently
W6–10	Worklet 1, 2, 3 evaluations together (roughly every two weeks)	Worklet reviews — (E) 19 Sep, 3 & 17 Oct · 10% each.
W11	Test, Consolidate & integrate	Integration & Validation review — (E) 24 Oct · 20%.
W13	Final demonstration + individual viva	Final demo / viva — (E) 4–6 Nov · 30%.

7. Deliverables

Across the semester your group submits and demonstrates the following. Each worklet is a working, demonstrable build — not just code.

- A signed project plan and a working development environment (week 1).
- A requirements note and a high-level design, including interface definitions for each worklet, agreed across the groups in your mini project (week 2).
- Three worklet builds, each demonstrated and reviewed (weeks 4–11).
- An integrated graph-algorithms explorer that combines the three worklets, tested and integrated (weeks 13–14).
- A final demonstration with an individual viva (week 13).
- A final report with your design notes and analysis (week 14).

8. How You Will Be Evaluated

Assessment is continuous: your work is scored at each evaluation point (E) in the timeline against a fixed rubric, and several criteria are assessed per student so individual understanding shows through. Each row below is one evaluation point; the percentages in that row show how it is split across the criteria (they total 100%), and the weight beside the week is that evaluation's share of your final grade.

Review	Conceptual Understanding (per student)	Analysis / Alternatives	Foundations First	Robustness / Depth / Correctness	Code Modularity / Organization	Documentation (per student)	Integration / Demo
W3 · 20	25%	25%		25%		25%	
W6 · 10			20%	50%	20%	10%	
W8 · 10			20%	50%	20%	10%	
W10 · 10			20%	50%	20%	10%	
W11 · 20	25%	25%					50%
W13 · 30	25%	25%		25%			25%

9. Tips for Success

- Get the smallest thing that runs working first, then iterate one task at a time.
- Keep a short design journal — the decisions and trade-offs you make are exactly what the viva asks about.
- Test edge cases early; a demo that handles the hard input (a disconnected graph, a negative-weight edge, a graph with a cycle) beats one that only handles the easy one.
- Divide the work, but make sure every member understands the whole system — the viva is individual.
- Whenever you reach for a library, ask whether it hides a concept you are meant to build yourself.

10. Common Pitfalls to Watch For

These mistakes are easy to make and easy for a reviewer to spot. Catching them early keeps your project correct and your viva answers convincing.

- Running Dijkstra on a graph with negative edges and trusting the wrong answer.
- Using an adjacency matrix for a sparse graph and paying $O(V^2)$ where a list would be linear.
- A union-find without path compression, so Kruskal is slower than it should be.
- Detecting cycles with a visited set that cannot tell a back edge from a cross edge.
- Reporting BFS as shortest-path on a weighted graph, where it does not hold.

Remember: A tool that runs BFS is not the point. Choosing the right representation, knowing when Dijkstra fails, and proving your bipartite or MST results are what make this a strong, well-understood project — and they are exactly what the individual viva will probe.